

크롤러 · 수집 요청 큐 · 백엔드 · 관리자 대시보드 전체 동작 프로세스

작성일: 2026-08-17

대상: 프로젝트를 처음 보거나, "수집 요청을 하면 실제로 무슨 일이 일어나는지"를 끝까지 추적하고 싶은 개발자

전제: [Purchase Research Agent 시스템 구조](#)와

[현재 수집 데이터와 DB 저장 흐름](#)이 컴포넌트 단위 설명이라면,

이 문서는 그 컴포넌트들을 관통하는 **하나의 요청이 처음부터 끝까지 어떤 코드를 타고 흐르는지를**

파일경로: [출번호](#) 단위로 추적한 문서다. 2026-08-16에 추가된 신선도 캐시/사전 재수집 스케줄러/

대량 재검증 비동기 배치/관리자 상품 재검증 화면까지 반영한다.

0. 전체 그림

```
[관리자 브라우저]
| ① 새 수집 요청 폼 제출 (또는 사용자 질문 → MCP → CollectionRefreshService)
▼
[Next.js purchase-web] app/admin/collections/new, app/api/collection-tasks
| ② 서버 route가 Product Backend REST 호출
▼
[Spring Boot Product Backend] CollectionTaskController → CollectionTaskPublisher
| ③ CollectionTask v1 JSON을 RabbitMQ에 발행 (publisher confirm 대기)
▼
[RabbitMQ] purchase-research.collection.v1 (exchange)
| ④ purchase-research.collection.search.v1 큐에서 대기
▼
[Python Collector Worker] RabbitCollectionWorker → CollectionTaskProcessor
| ⑤ ABC마트/29CM 실제 크롤링 (JSON+HTML 교차검증)
| ⑥ CollectionResult 봉투로 변환해 결과 큐에 발행 (running → success/partial/failed)
▼
[RabbitMQ] purchase-research.collection.result.v1
| ⑦ Spring Boot가 소비
▼
[Spring Boot] CollectionResultMessageService → CollectorResultStoreService
| ⑧ 하나의 transaction으로 PostgreSQL 저장 (products/merchant_products/offer_snapshots/ ...)
| ⑨ collection_jobs/collection_tasks 상태 갱신 (QUEUED→RUNNING→SUCCESS/FAILED)
▼
[PostgreSQL]
| ⑩ GET /internal/v1/products, /internal/v1/collection-jobs 로 조회
▼
[Next.js 관리자 대시보드] /admin/collections, /admin/collections/products
```

배경에서는 두 가지가 독립적으로 이 파이프라인의 ③을 트리거한다.

- **CollectionFreshnessScheduler**: TTL 지난 검색 조건을 낮은 우선순위로 자동 재수집 (§7)
- **BulkOfferVerificationRunner**: 관리자가 상품 재검증 화면에서 고른 상품들을 배경 스레드에서 순차적으로 재검증 Queue에 발행 (§9, 가장 최근 기능)

1. 관리자가 새 수집 요청을 등록하는 순간

1-1. 프론트엔드 폼

[frontend/purchase-web/app/admin/collections/new/new-collection-request-form.tsx:8-71](#)

판매처/검색어/시작페이지/페이지수/필터(가격, 사이즈, 색상, 카테고리, 재고보유)를 입력받는

순수 HTML `<form method="POST" action="/api/collection-tasks">` 다. 클라이언트 JS의 fetch가

아니라 **진짜 브라우저 폼 제출**을 쓴다 — 확장 프로그램/보안 소프트웨어가 fetch/XHR을

가로막아도 동작하게 하려는 설계다 (같은 파일 5-6줄 주석).

1-2. Next.js server route

[frontend/purchase-web/app/api/collection-tasks/handler.ts:63-119](#) 의

`handleCollectionTaskCreateRequest` 가 요청을 받는다.

- Content-Type 이 `application/x-www-form-urlencoded / multipart/form-data` 면 폼 데이터를 `buildPayloadFromFormData` (17-53줄)로 JSON 바디로 변환한다. 쉼표 구분 문자열(`sizes`, `colors`, `categories`)은 `parseCommaList` 로 배열화된다.
- `PRODUCT_BACKEND_BASE_URL` (기본 `http://127.0.0.1:8080`)의 `/internal/v1/collection-tasks/pages` 로 POST한다 (84줄). 즉 관리자 화면은 **항상 페이지**

단위 다중 등록 API를 쓴다(단일 등록 API `POST /internal/v1/collection-tasks` 는 Swagger나 다른 클라이언트용으로 남아 있다).

- 품 제출이면 결과를 JSON으로 반환하지 않고 `/admin/collections/new?created=<jobId>` 또는 `?error=...` 로 303 리다이렉트한다(12-15줄, 106-113줄) — 브라우저가 진짜 페이지 이동을 하게 만들어 새로고침 시 재제출 문제를 피한다.

1-3. Spring Boot 수집 작업 발행

`CollectionTaskController.publishPages`
(`services/product-backend/.../collection/controller/CollectionTaskController.java:87-122`)가 `BulkCollectionTaskRequest` 를 받아 `CollectionTaskPublisher.publishPages` 를 호출한다.

`CollectionTaskPublisher.publishPages`
(`.../collection/service/CollectionTaskPublisher.java:103-146`):

1. `startPage / pageCount` 가 1~200 범위(`MAX_PAGE=200`)를 벗어나면 `InvalidCollectionTaskException`.
2. 공통 `jobId = "job-" + UUID`, `requestedAt` 을 만들고 `startPage` 부터 `endPage` 까지 페이지마다 `createTask()` 로 독립된 `CollectionTaskMessage` 를 만든다(114-117줄). 한 페이지 = 한 RabbitMQ 메시지다.
3. `collectionJobService.register(tasks)` 로 발행 전에 모든 task를 `QUEUED` 상태로 PostgreSQL에 먼저 저장한다(118줄) — RabbitMQ가 잠깐 죽어도 "무엇을 요청했는지"는 항상 DB에 남는다.
4. 각 task를 순서대로 `publishTask()` 로 발행한다. 하나라도 broker confirm에 실패하면 남은 task들을 `markPublishFailed` 로 `PUBLISH_FAILED` 처리하고 예외를 던진다(126-134줄) — 부분 발행 실패를 명시적으로 취급한다.

`createTask()` (205-255줄)는 다음을 한다.

- `page` 가 1~200 범위 검증.
- `filters` 를 `normalizeFilters()` 로 정규화하고 `priceMin > priceMax` 면 예외.
- **멱등성 키(idempotencyKey)**를 `createIdempotencyKey()` (366-380줄)로 계산한다:
`{merchant, operation:"search", payload}` 를 key 정렬한 canonical JSON으로 만들어 SHA-256 해시 → `"collection:v1:" + hex(64자리)`. 같은 판매처+검색조건은 항상 같은 키를 갖는다(다만 이 키를 실제로 중복 차단에 쓰는 코드는 이 파일에서 확인되지 않았고, README의 "현재 미구현: 여러 검색어/여러 페이지 batch 작업" 항목과 일치).
- `priority` 기본값 20, `maxAttempts` 기본값 2(`DEFAULT_PRIORITY=20`, `DEFAULT_MAX_ATTEMPTS=2`, 48-51줄).

`publishTask()` (169-196줄)는 `CorrelationData` 를 사용해 **publisher confirm**을 5초(`CONFIRM_TIMEOUT_SECONDS=5`) 안에 기다린다. broker가 ACK 안 하거나(`!confirm.ack()`) mandatory 메시지가 라우팅 실패로 반송되면(`correlationData.getReturned() != null`) `CollectionTaskPublishException` 을 던진다 — "메시지를 보냈다"가 아니라 "broker가 받았다고 확인했다"까지 확인한 뒤에야 성공으로 간주한다.

2. RabbitMQ 큐 토폴로지 전체 목록

Go/Python/Spring Boot 세 서비스가 정확히 같은 이름을 공유한다(Python: `purchase-research-agent/app/messaging/rabbitmq.py:23-35`, Spring: `.../collection/messaging/CollectionQueueNames.java:8-19`).

종류	이름	비고
Exchange (direct, durable)	<code>purchase-research.collection.v1</code>	검색 작업/결과/retry가 모두 이 exchange를 통해 라우팅
Exchange (direct, durable)	<code>purchase-research.collection.dlx.v1</code>	Dead-letter 전용 exchange
Queue	<code>purchase-research.collection.search.v1</code>	Python/Go Worker가 소비하는 검색 작업 큐. <code>x-max-priority: 100</code> (우선순위 큐), dead-letter는 DLQ로
Queue	<code>purchase-research.collection.search.retry.v1</code>	재시도 대기 큐. <code>x-message-ttl: 5000</code> (5초) 뒤 자동으로 원래 큐(<code>SEARCH_ROUTING_KEY</code>)로 dead-letter되어 되돌아감 — TTL을 재시도 지연으로 악용하는 패턴

종류	이름	비고
Queue	<code>purchase-research.collection.search.dlq.v1</code>	재시도 소진/치명적 실패 작업의 최종 종착지
Queue	<code>purchase-research.collection.result.v1</code>	Spring Boot가 소비하는 결과 큐 (running/success/partial/failed 상태 메시지 전부)
Queue	<code>purchase-research.collection.result.dlq.v1</code>	결과 처리 실패의 DLQ

Routing key: `collection.search` (정상), `collection.search.retry`, `collection.search.dead`, `collection.result`, `collection.result.dead` (`rabbitmq.py:31-35`).

발행자/소비자 매트릭스

큐	발행자	소비자
<code>search.v1</code>	Spring <code>CollectionTaskPublisher</code>	Python <code>RabbitCollectionWorker</code> (Go Worker도 같은 큐를 소비 가능 — 전환 검증 중에는 하나만 실행)
<code>search.retry.v1</code>	Python Worker 자신(<code>decide_message</code> 가 retryable 실패 시 재발행)	없음(소비 안 함, TTL 뒤 자동 dead-letter되어 <code>search.v1</code> 로 복귀)
<code>result.v1</code>	Python/Go Worker	Spring <code>CollectionResultConsumer</code> → <code>CollectionResultMessageService</code>

2-1. Python Worker의 메시지 처리 순서 (`decide_message`)

`purchase-research-agent/app/messaging/rabbitmq.py:55-103`

1. `decode_collection_task(body)`로 JSON Schema + 의미 검증(`requestedAt`이 timezone 포함 ISO 8601인지, `attempt < maxAttempts`인지 등, `contracts.py:74-116`).
2. Schema 자체가 깨졌으면 `extract_collection_task_identity()`로 `taskId/jobId`만이라도 안전하게 복구를 시도하고(정규식 `^[A-Za-z0-9][A-Za-z0-9._-]{0,127}$`, `contracts.py:27,119-143`), 복구되면 계약 위반 실패 결과를 결과 큐에 발행 후 원본은 `reject` (requeue 안 함) — Spring이 “왜 실패했는지” 알 수 있게 하면서도 원본 잘못된 메시지를 무한 루프시키지 않는다.
3. 정상 디코딩되면 `processor.process(task)` 실행(→ §4).
4. 결과 `status == "failed"`이고 `error.retryable == True`이고 `task.can_retry()` (`attempt+1 < maxAttempts`)면 **attempt를 1 증가시킨 새 task를 retry 큐로 재발행**하고 원본은 `ack` (성공 취급 — 재시도로 이관했으므로).
5. 그 외에는 결과를 `result` 큐로 발행. 최종 `failed`면 원본 메시지는 `reject` (DLQ 행), `success` / `partial`이면 `ack`.

`RabbitCollectionWorker._handle_message` (`rabbitmq.py:155-204`)는 메시지를 받자마자 먼저 **running 상태 결과를 즉시 발행**한다(`_running_task_result`, 356-378줄) — Spring이 이걸 받으면 `collection_tasks.status`를 `RUNNING`으로 바꿔 대시보드에서 “지금 처리 중”이 보이게 한다. 이후 실제 처리 결과(성공/실패)를 다시 발행한다. 즉 **task 하나당 결과 메시지가 최소 2번(running + 최종) 발행**된다.

`declare_topology()` (`rabbitmq.py:250-309`)는 모든 큐/exchange를 멍등하게 선언한다 (`durable=True`, `prefetch_count=1` — Worker instance 하나가 동시에 메시지 1개만 처리, `publisher_confirms=True` + `on_return_raises=True`로 Worker의 발행도 Spring 발행과 동일한 안전성 수준을 유지).

3. Python 크롤러 내부 동작

3-1. 진입점: `CollectionTaskProcessor.process`

`purchase-research-agent/app/messaging/processor.py:67-156`

1. `_validate_supported_search()` (215-230줄)로 `locale=ko-KR`, `currency=KRW`만 지원함을 확인하고, `filters.attributes`가 있으면 즉시 실패(Python은 아직 미지원).
2. `rate_limiter`가 설정돼 있으면(Redis 기반, §3-5) `acquire(merchant)`로 판매처 전역 토큰을 기다린다. 타임아웃 나면 `RATE_LIMIT_TIMEOUT`으로 즉시 실패(`retryable=True`).
3. `asyncio.wait_for(crawler.search_items(...), timeout=30.0)`로 **작업 하나에 최대 30초만 허용**(`timeout_seconds` 기본값). `max_items=50`, `detail_limit=0`, `page=payload["page"]`, `max_pages=1` — 큐로 들어오는 작업 하나는 정확히 페이지

1개, 상품 최대 50개로 작게

고정된다. `detail_limit=0` 이므로 큐 경로에서는 리뷰/옵션 상세를 조회하지 않는다
(검색 결과 필드만).

4. `_apply_filters()` (233-289줄)로 Queue의 `priceMin/priceMax/categories/sizes/colors/ inStockOnly` 를 크롤러가 실제로 확인한 원본 필드만 사용해 AND 조건으로 후처리 필터링한다
(판매처 API가 필터를 지원 안 해도 Python 쪽에서 재확인).
5. 결과 상품이 0개인데 경고가 있으면(`not products and warnings`) 첫 경고 메시지를 실패 사유로 승격하고, `_is_retryable_message()` (208-212줄)로 `timeout/429/5xx/connection` 등의 패턴이면 retryable로 표시.
6. 성공 시 `build_collector_result()` (§6)로 Spring Boot 계약(camelCase)으로 변환.
7. warnings 가 있으면 `status="partial"`, 없으면 `"success"`.

예외 매핑: `asyncio.TimeoutError` → `COLLECTOR_TIMEOUT` (retryable),
`ValueError` → `INVALID_TASK` (non-retryable), 그 외 `Exception` → `COLLECTOR_UNAVAILABLE`
(retryable) — `processor.py:135-154`.

3-2. ABC마트: JSON 검색 + HTML 렌더링 교차 검증

`purchase-research-agent/app/crawlers/abcmart/crawler.py:277-382`

ABC마트는 검색 결과를 두 가지 다른 소스로 동시에 가져와서 서로 비교한다.

1. `AbcJsonFetcher.fetch_page()` — 판매처 검색 JSON API(내부 API)에서 상품 목록을 받는다.
2. `AsyncWebCrawler` (`crawl4ai`, 실제 브라우저 렌더링)로 같은 검색 조건의 HTML 페이지를 `delay_before_return_html=2.0` 초 대기 후 가져와 BeautifulSoup으로 파싱한다
(`_parse()`, 545-606줄: `li.col-list-item.prod-item` 을 CSS 선택자로 파싱).
3. `reconcile_page()` (`verification.py`, import는 14줄)로 JSON 결과와 HTML 결과를 상품 ID(`prdtNo`) 단위로 대조해 `MATCHED/MISMATCH/MISSING_IN_HTML/MISSING_IN_JSON` 등의 검증 상태를 만든다. HTML 로드가 실패하면 `mark_failed()` 로 검증 자체를 실패 처리.
4. 페이지 요청 실패 시 한 번 재시도(2초 대기 후, `attempt in range(2)`, 321-370줄) —
"2026-08-06 실측: 동시 워커 부하 중 순간적 연결 예외가 재시도 없이 pagination을 통째로 끊어 목표치의 88%를 날린 사례"라는 주석이 재시도를 추가한 이유를 설명한다(318-320줄).
5. 검색 URL은 `channel=10001, sort=point, tabGubun=total` 등 고정 파라미터(`_search_url()`, 384-398줄).

카테고리/브랜드 크롤링(2026-08-16 신규): `crawl_category()` (400-408줄)와

`crawl_by_brand()` (463-471줄)는 `CATEGORIES / BRANDS` 디렉터리(모듈 상단, 39-264줄)에서 (`ctgrNo`, `genderGbnCode`) 또는 (`brandNo`, `tChnnlNo`)를 찾아 목록 URL을 순회한다. 이 두 디렉터리는 "2026-08-15에 검색 결과 페이지 HTML에 박힌 nav를 파싱해 채운 목록"이라는 주석이 있고(32-38줄, 122-128줄), **완전한 커버리지를 보장하지 않는다**고 명시돼 있다(JS 지연 로드 하위 메뉴가 nav 스냅샷에 안 잡혔을 수 있음). 카테고리/브랜드 크롤링은 검증(JSON/HTML 교차검증)을 하지 않고 HTML 파싱 결과만 사용한다.

3-3. 29CM: JSON 목록 + 상세 HTML(JSON-LD) 검증

`purchase-research-agent/app/crawlers/cm29/crawler.py:143-215`

1. `display-bff-api.29cm.co.kr` 의 listing API에 POST(`_LISTING_URL`, 12줄) —
`sortType: RECOMMENDED`, `pageRequest: {page, size:50}`.
2. `ensure_success()` (`access_safety.py`)로 상태코드 분류 후 `_parse()` (294-337줄)로 공통 필드 변환. 29CM 목록 API는 **색상/사이즈 옵션과 style code를 카드 단위로 주지 않는다**
(328-329줄 주석).
3. `verify_products()` (`verification.py`)로 선택된 모든 상품의 상세 페이지 HTML에 포함된 `Product` JSON-LD를 가져와 목록 JSON과 비교 검증한다(195-201줄) — ABC마트와 달리 페이지가 아니라 **상품 단위로 검증**.
4. **카테고리 브라우징 API가 없다**(21-28줄 주석): `pageType=CATEGORY_PLP` 를 넣어도 무시되고 전체 카탈로그(`total=1,086,031`)가 그대로 반환됨을 실측 확인. 그래서 `crawl_category()` (227-274줄)는 대표 검색어로 검색한 뒤 `largeCategoryName/middleCategoryName` 이 정확히 일치하는 상품만 사후 필터링한다(`_matches_category()`, 359-374줄, 부분 문자열이 아니라 > 로 나눈 단계 단위 정확 일치). `max_pages=40` 상한.
5. **legacy URL redirect 차단 수정**(2026-08-16, 커밋 36792be): 상품 상세 조회 시 구 URL 형식이 redirect를 유발해 `access_safety.ensure_success()` 의 `MERCHANT_REDIRECT_BLOCKED` (300~399 상태코드는 자동 추적하지 않고 즉시 차단, non-retryable, `access_safety.py:38-43`)로 막혔던 문제를 해결한 커밋이다.

3-4. 접근 안전성 공통 처리 (access_safety.py)

ensure_success() (24-66줄)가 모든 판매처 HTTP 응답을 분류한다.

상태	오류 코드	retryable
3xx	MERCHANT_REDIRECT_BLOCKED	아니오 — redirect를 자동 추적하지 않는다 (원칙: 접근 통제 비우회)
401/403	MERCHANT_ACCESS_BLOCKED	아니오
429	MERCHANT_RATE_LIMITED	예
5xx	MERCHANT_TEMPORARY_FAILURE	예
그 외 비2xx	MERCHANT_HTTP_FAILURE	아니오

safe_exception_message() (69-87줄)는 timeout/connection/기타 예외를 URL이나 응답 body, traceback 없이 안전한 문자열로만 변환한다 — 로그나 큐 메시지에 민감정보가 남지 않게 하는 설계.

3-5. 판매처 전역 rate limit (Redis token bucket)

purchase-research-agent/app/services/rate_limiter.py

- CollectionRunner.min_interval_seconds 나 jittered_sleep 은 한 프로세스 안에서만 유효. Worker를 여러 instance로 띄우면 판매처 입장에서 본 전체 요청 속도를 제어할 수 없다 (1-11줄 문제 설명).
- MerchantRateLimiter (122-191줄)가 Redis Lua 스크립트(_TOKEN_BUCKET_SCRIPT , 24-55줄)로 판매처별 token bucket 하나를 여러 프로세스가 원자적으로 나눠 쓰게 한다. Redis는 싱글스레드로 스크립트를 실행하므로 레이스 컨디션이 없다.
- create_rate_limiter_from_env() (103-119줄): PURCHASE_RESEARCH_REDIS_URL 환경변수가 있을 때만 활성화(옵트인). PURCHASE_RESEARCH_RATE_LIMIT_PER_SECOND 기본값 1.0 req/s.
- acquire() 가 max_wait_seconds (기본 30초) 안에 토큰을 못 얻으면 RateLimitTimeoutError → processor.py 에서 RATE_LIMIT_TIMEOUT 실패로 변환.

3-6. 상세 조회 신선도 캐시 (2026-08-16 신규)

purchase-research-agent/app/services/detail_freshness_cache.py

Queue 기반 운영 경로(RabbitMQ Worker)는 이 캐시를 쓰지 않는다 — 파일 상단 주석(1-8줄)에 명시: Queue 작업은 이미 상품 50개/30초 제한으로 작고, "다시 수집해야 하는가" 판정은 Spring Boot CollectionRefreshService 가 검색어 단위로 담당한다. 이 캐시는 scripts/ run_unified_crawl.py 같은 수동 대량 실행 스크립트 전용 보조 도구다.

- DetailFreshnessCache(path, ttl_hours=24.0) : 판매처별 {상품ID: 마지막 상세조회 시각} 을 로컬 JSON 파일에 저장.
- split_stale(merchant, products) (44-69줄): ttl_hours 안에 조회한 적 있으면 fresh 로, 아니면(또는 source_product_id 가 없으면 무조건) stale 로 분류해 반환.
- mark_fetched() (71-85줄): 이번엔 상세를 새로 조회한 상품들의 시각을 갱신하고 즉시 저장 (원자적 교체, os.replace).
- DB에 저장된 실제 값은 기억하지 않는다 — "언제 조회했는지"만 기록하므로, 캐시로 건너뛴 상품은 이번 실행 결과에서 검색 단계 필드만 채워지고 상세(리뷰/옵션/평점)는 비어 있다. 직전 결과 파일과 병합하는 것은 호출자 책임(10-12줄).

3-7. CollectorService / CollectionRunner — Queue 밖의 대량 수집 경로

services/crawler_service.py 가 판매처 등록(SITE_CRAWLERS), 검색/카테고리/브랜드 수집, 상세 첨부(attach_details), ABC마트 결과의 계약 검증(validate_items)을 연결한다. search_multi_site() (65-94줄)는 asyncio.gather(... , return_exceptions=True) 로 여러 판매처를 동시에 검색하고 한 판매처 예외가 다른 판매처에 전파되지 않게 격리한다.

services/collection_run.py 의 CollectionRunner 는 Queue 경로와 별개의 안전장치다 (1-8줄 docstring): 스크립트/수동 대량 수집처럼 페이지 수를 미리 알 수 없는 경로 전용.

- CollectionRunConfig : max_items (≤10,000), page_size (≤100), request_budget (페이지 조회 시도 상한), min_interval_seconds , max_wall_seconds (총 시간 상한), resume .
- 중단 사유(stop_reason)를 명시적으로 기록: target_reached , request_budget_exhausted , wall_time_exhausted , http_429 , http_401_403 , no_more_results , rate_limit_timeout 등(193-249줄) — 왜 멈췄는지 항상 설명 가능하게 설계.

- checkpoint/누적 상품을 JSON으로 원자적 저장(`_write_json`, 320-325줄: `.tmp` 파일 쓰고 `os.replace`)해 **resume** 가능.

4. 결과가 다시 Spring Boot로 돌아와 저장되는 과정

4-1. 계약 검증

```
CollectionResultMessageService.process()  
( ... /collection/service/CollectionResultMessageService.java:54-74 )
```

1. `objectMapper.readValue(body, CollectionResultEnvelope.class)` 로 역직렬화 실패 시 `InvalidCollectionResultMessageException`.
2. Jakarta Bean Validation(`validator.validate(envelope)`)으로 필드 제약 검사.
3. `envelope.validateSemantics()` 로 상태별 의미 검증(Python `contracts.py` 의 `validate_collection_result_envelope` 와 대칭되는 Java 쪽 검증).
4. `status` 에 따라 분기:
 - "running" → `collectionJobService.recordRunning(envelope)` (task를 `RUNNING` 으로)
 - "failed" → `collectionJobService.recordFailed(envelope)`
 - 그 외(success/partial) → `collectorResultStoreService.store(...)` 로 실제 저장 후 `collectionJobService.recordCompleted(envelope, report.productCount())`

전체가 `@Transactional` 이므로 저장과 job 상태 갱신이 원자적이다.

4-2. 실제 저장 — `CollectorResultStoreService.store()`

```
... /collection/service/CollectorResultStoreService.java:96-126
```

한 `CollectorResult` (최대 50개 상품)를 하나의 transaction으로 처리한다.

1. Bean Validation + `result.validateStorable()`.
2. `storeSearchContext(result)` (182-200줄): `operation == "search"` 일 때만, `collection_search_contexts.request_id` 가 이미 있으면 같은 **merchant/query/filters**인지 확인하고 다른면 `UnstorableCollectorResultException` ("같은 requestId를 다른 검색 조건으로 재사용할 수 없습니다") — 없으면 새로 저장.
3. 상품마다 순서대로:
 - `upsertMerchantProduct()` (209-236줄): (`merchant, externalId`) 로 기존 상품을 찾아 있으면 `Product.update()` (이름/브랜드/카테고리/이미지 갱신) + `updateCollection()` (`product_url`, `last_collected_at` 갱신), 없으면 `Product` 와 `MerchantProduct` 를 새로 생성. **상품 행을 무한 복제하지 않는다.**
 - `saveOfferSnapshot()` (246-267줄): 가격/배송비/재고/평점/리뷰수/출처를 **항상 새 행으로 추가**(과거 값을 덮어쓰지 않음 — \$6 "왜 snapshot인가").
 - `saveOptions()` (276-294줄): snapshot에 딸린 사이즈/색상 옵션 각각을 저장.
 - `saveEvidence()` (304-315줄): 상품 출처 1건 + (배송 출처가 상품 출처와 다르면) 배송 출처 1건을 `evidence` 에 저장.
 - `saveVerification()` (146-173줄): Python이 보낸 `verification` (JSON/HTML 비교 결과)이 있으면 `product_verifications` 에 저장.
 - `searchDocuments` 에 `merchantProduct.getId()` 와 `buildSearchDocument()` (129-136줄: 상품명/브랜드/카테고리/검색어/판매처를 " / " 로 합친 문자열)를 누적.
4. 상품 처리가 끝나면 `ApplicationEventPublisher` 로 `ProductSearchDocumentsChanged` 이벤트를 발행(118줄) — **전문 검색/임베딩 인덱스 갱신은 같은 transaction이 아니라 이벤트 기반으로 분리**돼 있다(commit 후 리스너가 처리하는 구조로 추정, 이 문서의 조사 범위 밖).

4-3. Job/Task 상태 머신 — `CollectionJobService`

```
... /collection/service/CollectionJobService.java
```

- 등록(`register`, 60-77줄): RabbitMQ 발행 전에 job을 `QUEUED`, 모든 task를 `QUEUED` 로 먼저 저장.
- 발행 실패(`markPublishFailed`, 85-100줄): 실패한 task들을 `PUBLISH_FAILED` 로 표시하고 `refreshJobStatus()` 재계산.

- **RUNNING 기록**(recordRunning, 129-141줄): Worker가 running 결과를 보내면 처음 RUNNING으로 바뀔 때만(task.start()가 changed=true 반환할 때만) job 상태를 재계산 — 중복이나 최종 결과 뒤에 늦게 온 시작 이벤트는 무시.
- **완료/실패 기록**(recordCompleted / recordFailed, 110-159줄): validateJobIdentity()로 결과의 jobId가 등록된 task의 상위 job과 일치하는지 검사(다른 job 결과가 섞이는 것 방지).
- **job 상태 재계산**(refreshJobStatus, 270-300줄):
- 아직 끝나지 않은 task가 있으면: 하나라도 RUNNING이면 job도 RUNNING, 아무것도 안 끝났으면 QUEUED, 일부만 끝났으면 PROCESSING.
- 모든 task가 종료됐으면: 전부 SUCCESS면 COMPLETED, 전부 FAILED / PUBLISH_FAILED면 FAILED, 섞여 있으면 PARTIAL.

job 상태값: QUEUED → RUNNING/PROCESSING → COMPLETED | PARTIAL | FAILED

(V9 마이그레이션으로 RUNNING이 추가됨, V9__add_collection_running_status.sql:1-7).

task 상태값: QUEUED → RUNNING → SUCCESS | PARTIAL | FAILED | PUBLISH_FAILED.

4-4. Job 재실행 (retry)

CollectionJobController.retry(CollectionJobController.java:120-142) →

CollectionTaskPublisher.retryJob() (158-161줄) →

CollectionJobService.toRetryRequest() (201-223줄): FAILED / PARTIAL job만 재실행 가능(RETRYABLE_STATUSES)하며, 등록 당시 저장해둔 request_snapshot (V11 마이그레이션, collection_jobs.request_snapshot JSONB 컬럼)에서 원래의 merchant/query/limit/locale/currency/filters를 복원해 새 jobId로 publishPages()를 다시 호출한다 — 완전히 새로운 job이 생성되며 기존 job의 이력은 그대로 남는다.

5. DB 스키마 전체 (Flyway V1~V12)

services/product-backend/src/main/resources/db/migration/

5-1. 상품·수집 결과 (V1, V2, V3)

```
products -- 판매처 무관 최소 상품 정보
  id, name, brand, category_path(jsonb), image_urls(jsonb), created_at, updated_at

merchant_products -- (merchant, external_id) 단위 판매처 상품
  id, product_id → products, merchant, external_id, product_url,
  last_collected_at, created_at, updated_at
  UNIQUE(merchant, external_id)

offer_snapshots -- 수집 시점의 가격/재고 이력 (덮어쓰지 않고 계속 추가)
  id, merchant_product_id → merchant_products, request_id,
  price_amount, currency, shipping_fee_amount, shipping_fee_currency, shipping_summary,
  stock_status, rating, review_count, source_url, collected_at, collector_version,
  created_at
  CHECK price_amount ≥ 0, CHECK rating BETWEEN 0 AND 5

product_options -- snapshot 시점의 사이즈/색상/옵션별 재고
  id, offer_snapshot_id → offer_snapshots, external_id, label, size, color,
  stock_status, price_amount, currency, source_url, collected_at, collector_version

evidence -- 값의 공개 출처(URL/시각/collector 버전)
  id, merchant_product_id, offer_snapshot_id, evidence_type, source_url,
  collected_at, collector_version

collection_search_contexts -- requestId 하나당 검색어/필터를 한 번만 저장 (V2)
  request_id (PK), merchant, search_query, filters(jsonb), collected_at,
  collector_version

product_verifications -- JSON/HTML 교차검증 결과 (V3)
  id, merchant_product_id, offer_snapshot_id, status
  CHECK status IN (PENDING, MATCHED, MISMATCH, MISSING_IN_HTML, MISSING_IN_JSON, FAILED),
  compared_fields(jsonb), differences(jsonb), json_source_url, html_source_url, verified_at
```

5-2. 수집 job/task 추적 (V4, V9, V11)

```
collection_jobs
  job_id (PK), merchant, operation, search_query, status, total_tasks,
```

```
start_page, end_page, requested_at, completed_at, request_snapshot(jsonb, V11)
CHECK status IN (QUEUED, RUNNING[V9], PROCESSING, COMPLETED, PARTIAL, FAILED)
```

```
collection_tasks
task_id (PK), job_id → collection_jobs, page, status, product_count,
verification_total/matched/mismatched/failed/missing_in_html/missing_in_json/pending,
requested_at, started_at, completed_at, duration_ms,
error_code, error_message, error_retryable
CHECK status IN (QUEUED, RUNNING[V9], SUCCESS, PARTIAL, FAILED, PUBLISH_FAILED)
UNIQUE(job_id, page)
```

5-3. 구매 전 재검증 (V10)

```
offer_verification_requests
verification_id (PK, UUID), merchant_product_id → merchant_products,
job_id → collection_jobs, baseline_price_amount, baseline_currency,
baseline_stock_status, baseline_collected_at, requested_at
```

재검증도 결국 `collection_jobs / collection_tasks` 를 그대로 재사용한다 — 재검증 요청은 "우선순위 100짜리 일반 검색 CollectionTask"로 발행되고(\$8), 그 결과가 이 테이블의 `baseline_*` 값과 비교된다.

5-4. Agent Run (V12, 참고 — 이 문서의 핵심 범위 밖)

```
agent_runs -- 상태 기반 구매 조사 실행 (research_session 1:1)
run_id (PK, UUID), research_session_id (UNIQUE) → research_sessions,
status, verification_id → offer_verification_requests, error_code, error_message,
version (낙관적 락)

agent_run_events -- 사용자에게 공개할 단계별 진행 사건 (append-only)
event_id, run_id → agent_runs, sequence_no, status, event_type, message
UNIQUE(run_id, sequence_no)

agent_run_collection_jobs -- Agent Run과 그 실행이 발행한 수집 job의 연결
run_id, job_id → collection_jobs, merchant, data_status
PRIMARY KEY(run_id, job_id)
```

5-5. 관계도

```
collection_search_contexts
└─ request_id
   ↓ 같은 request_id
offer_snapshots.request_id
products
└─ merchant_products (merchant + external_id UNIQUE)
   └─ offer_snapshots (계속 추가, 덮어쓰지 않음)
      └─ product_options
         └─ product_verifications
└─ evidence
└─ offer_verification_requests (baseline 스냅샷)

collection_jobs
└─ collection_tasks (job_id + page UNIQUE)
```

6. Python → Spring 데이터 변환 (collector_result_adapter.py)

`purchase-research-agent/app/services/collector_result_adapter.py`

`build_collector_result()` (33-90줄)가 Python 크롤러의 `snake_case` 원본 상품 목록을 Spring Boot의 `POST /internal/v1/collection-results` 입력 계약(camelCase)으로 변환한다.

- `_convert_product()` (138-178줄): `source_product_id/title/link` 중 하나라도 없으면 `ValueError`. `price` 는 `parse_won()` (15-30줄)으로 "19,000원" → 19000 정수 변환.
- `_convert_options()` (205-223줄): 크롤러가 반환한 `options.sizes / options.colors` 배열을 "사이즈 옵션"과 "색상 옵션"으로 개별 분리해 만든다 — 사이즈×색상 조합 재고까지는 아직 표현하지 않음(각 옵션 행은 사이즈 또는 색상 하나만 채움, 210-211줄 주석: "연결 관계가 없는 크롤러 옵션 목록을 과장하지 않고").

- `_stock_status()` (244-258줄): Python의 `in_stock: bool | None` 을 `available / out_of_stock / unknown` 3값으로 변환 — **모르면 재고 있다고 과장하지 않고 `unknown` 으로 명시.**
- `build_collector_result_batches()` (93-135줄): 대량 결과를 Spring이 요구하는 `batch_size (≤50)` 단위로 쪼갬다. `request_id_prefix-b001, -b002, ...` 처럼 순번을 붙인 `requestId`를 만들고, 크롤러 경고는 **첫 배치에만** 기록한다(132줄).

7. 신선도 기반 사전 재수집 스케줄러 (2026-08-16 신규)

7-1. 최신성 판정 — `CollectionRefreshService`

... /collection/service/CollectionRefreshService.java

`request()` (55-70줄): 특정 (`merchant, query`) 의 마지막 수집완료 시각을 `collection_search_contexts` 에서 조회해 상태를 판정한다.

```
staleBoundary = now - staleAfterHours(기본 24시간)
latestCollectedAt == null      → MISSING
latestCollectedAt < staleBoundary → STALE
그 외                          → FRESH
```

FRESH 이고 `force=false` 면 아무 것도 안 하고 기존 상태만 반환(`NO_ACTION`). 그 외에는 `REFRESH_LIMIT=5` 개 상품만, **우선순위 30**으로 `CollectionTaskPublisher.publish()` 를 호출한다 (66-67줄) — 이 반응형 갱신은 **사용자가 실시간으로 기다리는 경로**이므로 우선순위를 높게 둔다.

7-2. 배경 사전 갱신 — `refreshStaleBatch()`

같은 클래스 81-98줄. `staleBoundary` 를 지난 "기본 검색 범위"(필터 없는 순수 검색)를 `searchContextRepository.findStaleDefaultSearchContexts(staleBoundary, batchSize)` 로 **마지막 수집이 오래된 순으로** 최대 `batchSize` 개 찾아 **우선순위 5** (`BACKGROUND_REFRESH_PRIORITY, 31` 줄 — 반응형 갱신의 우선순위 30보다 낮음)로 발행한다. 개별 발행 실패는 로그만 남기고 나머지 `batch` 처리를 막지 않는다(`catch CollectionTaskPublishException, 92-95`줄).

7-3. 주기 실행 — `CollectionFreshnessScheduler`

... /collection/service/CollectionFreshnessScheduler.java

- `@ConditionalOnProperty(prefix = "purchase-research.freshness.scheduler", name = "enabled", havingValue = "true")` — 기본값 `false`, 명시적으로 커야만 동작(19줄, rate limiter처럼 옵트인 배경 작업).
- `@Scheduled(fixedDelayString = "... interval-ms:1800000", initialDelayString = "... initial-delay-ms:60000")` — 기본 **30분**마다, 첫 실행은 앱 기동 60초 후.
- `batch-size` 기본값 20(`@Value("${purchase-research.freshness.scheduler.batch-size:20}")`).
- 매 실행마다 `collectionRefreshService.refreshStaleBatch(batchSize)` 를 호출하고 발행 건수를 로그에 남긴다.

효과: 사용자가 실제로 검색하기 전에 오래된 검색 조건을 낮은 우선순위로 미리 갱신해 뒤서, 사용자가 왔을 때 `CollectionRefreshService.request()` 가 `STALE / MISSING` 을 반환하며 만드는 대기 시간을 줄인다.

8. 구매 전 재검증 (단건) — `OfferVerificationService`

... /evidence/service/OfferVerificationService.java

`request(productId)` (69-88줄):

1. `merchantProductRepository / productQueryService` 로 현재 저장된 **기준(baseline) snapshot**을 가져온다.
2. **VERIFICATION_LIMIT=5, 우선순위 100**(가장 높음)으로 상품명을 검색어 삼아 `CollectionTaskPublisher.publish()` 로 새 검색 작업을 발행한다 — 재검증도 특수 API가 아니라 **일반 검색 `CollectionTask`를 최고 우선순위로 발행하는** 것뿐이다.
3. `offer_verification_requests` 에 `baseline_price_amount/currency/stock_status/ collected_at` 을 저장.

4. 상태는 `QUEUED` 로 즉시 반환(202 Accepted가 아니라 `@ResponseStatus(HttpStatus.ACCEPTED)` 컨트롤러에서 202).

`get(verificationId)` (94-115줄): `collectionJobService.get(request.getJobId())` 로 job 상태를 확인해 `QUEUED` / `RUNNING` / `PROCESSING` 이면 그대로 반환, `FAILED` 면 `FAILED`. 완료됐으면 최신 `ProductSummary` 를 다시 조회해 `latest.source().collectedAt()` 이 `baselineCollectedAt` 보다 이후인지 확인(안 그러면 아직 새 결과가 반영 안 된 것으로 보고 `NOT_FOUND`). 이후 `changes()` (127-139줄)로 가격/통화/재고 상태만 비교해 변경된 필드 이름 목록(`price`, `stockStatus`)을 만들고 `VERIFIED` (변경 없음) 또는 `CHANGED` 를 반환.

9. 대량 재검증 비동기 배치 (2026-08-16, 가장 최근 기능)

관리자가 상품 재검증 화면에서 수백~수천 건을 한꺼번에 고를 수 있는데, 상품 하나당 \$8의 `request()` 가 RabbitMQ broker confirm(최대 5초)까지 기다리므로 **동기 처리하면 대량 요청 시 HTTP 요청이 수 분간 응답하지 않는다**. 이를 "접수(즉시 응답)"와 "처리(배경에서 순차 진행)"로 분리한 것이 이번 기능의 핵심이다.

9-1. 접수 — `OfferVerificationController.requestBulk`

... /evidence/controller/OfferVerificationController.java:69-76

```
POST /internal/v1/offer-verifications/products/bulk
body: { "productIds": [1, 2, 3, ...] } // 최대 10,000개(주석 61줄)
```

1. `batchTracker.create(request.productIds().size())` 로 배치 ID(UUID) 생성.
2. `bulkRunner.runAsync(batchId, request.productIds())` 호출 — **호출만 하고 결과를 기다리지 않는다**.
3. 즉시 `batchTracker.get(batchId)` (방금 만든 `PROCESSING` 상태)를 202 Accepted로 반환.

9-2. 배경 처리 — `BulkOfferVerificationRunner`

... /evidence/service/BulkOfferVerificationRunner.java

```
@Async("bulkVerificationExecutor")
public void runAsync(UUID batchId, List<Long> productIds) {
    for (Long productId : productIds) {
        try {
            var verification = offerVerificationService.request(productId); // $8과 동일 경로
            batchTracker.recordSuccess(batchId, productId, verification);
        } catch (ResponseStatusException e) {
            batchTracker.recordFailure(batchId, productId, e.getReason());
        } catch (CollectionTaskPublishException e) {
            batchTracker.recordFailure(batchId, productId, e.getMessage());
        }
    }
    batchTracker.complete(batchId);
}
```

- **상품을 순서대로 하나씩 처리**(병렬 아님) — 각 건이 \$8의 `OfferVerificationService.request()` 를 그대로 호출하므로 판매처 검색 `CollectionTask` 하나가 우선순위 100으로 발행된다.
- 상품이 없거나(`ResponseStatusException`) Queue 발행이 실패해도(`CollectionTaskPublishException`) **나머지 상품 처리를 막지 않는다** — 실패는 배치 결과에 기록만 되고 루프는 계속된다.
- `@Async` 가 실제로 별도 스레드에서 돌려면 **프록시를 통한 외부 호출**이어야 하므로, 배치를 만든 컨트롤러가 이 메서드를 직접 호출하며 이 클래스는 self-invocation하지 않는다 (파일 15-21줄 주석).

`AsyncConfig` (... /common/config/AsyncConfig.java)가 `bulkVerificationExecutor` 스레드풀을 정의: `corePoolSize=2`, `maxPoolSize=4`, `queueCapacity=50`. 배치 하나는 순차 처리라 스레드 1개로 충분하지만, **여러 배치가 동시에 접수될 수 있어** 소규모 풀을 둔다 (11-14줄 주석).

9-3. 진행 상태 추적 — `BulkVerificationBatchTracker`

... /evidence/service/BulkVerificationBatchTracker.java

- **메모리 저장소**(`ConcurrentHashMap<UUID, Batch>`) — 서버 재시작 시 배치 진행 기록 자체는 사라지지만, **이미 발행된 Queue 작업과 offer_verification_requests 저장은 durable하게 남는다**(19-24줄 주석). 즉 이 트래커는 어디까지나 "관리자 화면에 진행률을 보여주기 위한 보조 상태"이지 신뢰의 원천(source of truth)이 아니다.
- **Batch** (94-104줄): `requestedCount` (요청 건수), `startedAt`, `results` (`CopyOnWriteArrayList<Item>`, 스레드 안전), `completedAt` (volatile).
- `get(batchId)` (81-92줄): `status == completedAt == null ? PROCESSING : COMPLETED`, `processedCount = results.size()`, `queuedCount = results` 중 `success`인 것의 개수.

9-4. 진행 상태 조회

```
GET /internal/v1/offer-verifications/products/bulk/{batchId}
→ BulkVerificationBatchResponse { batchId, status, requestedCount, processedCount,
    queuedCount, results[], startedAt, completedAt }
```

배치가 없으면 404(`ResponseStatusException(NOT_FOUND)` , `OfferVerificationController.java:85-87`).

9-5. 관리자 화면에서의 흐름

`frontend/purchase-web/app/admin/collections/products/`

1. `ProductsRecheck` (`products-recheck.tsx`) 표에서 체크박스로 상품을 고르거나, "**조회된 N건 전체 재검증**" 버튼(74-81줄)으로 현재 필터 결과 전체를 한 번에 선택한다. 두 경우 모두 **일반** `<form method="POST" action="/api/offer-verifications/bulk">` (fetch 없음, 4-9줄 주석과 동일한 이유)이고 `productIds` 를 hidden input 배열로 보낸다.
2. Next.js route `app/api/offer-verifications/bulk/handler.ts` 의 `handleBulkOfferVerificationRequest()` (30-80줄)가 `productIds` 가 비어 있으면 즉시 `/admin/collections/products?error=...` 로 리다이렉트(37-43줄). 있으면 Product Backend의 `/internal/v1/offer-verifications/products/bulk` 를 호출하고, 성공하면 응답의 `batchId` 를 `/admin/collections/products?batchId=<id>` 로 리다이렉트(66-68줄).
3. `products/page.tsx` (13-75줄)가 `batchId` 쿼리 파라미터를 보면 `loadBatchStatus(batchId)` (`products-data.ts:51-64`)로 진행 상태를 가져와 배너로 보여준다: "요청 N건 중 M건 처리됨 · K건 큐 등록 성공". `PROCESSING` 이면 "새로고침" 링크를 보여줘 **수동 새로고침으로** 폴링하게 한다(자동 폴링 JS 없음 — 이 프로젝트 전반의 "fetch/XHR 없이 진짜 페이지 이동" 원칙과 일관).

10. 상품 조회 정렬 · 경과 시간 필터 (2026-08-16 신규)

`GET /internal/v1/products` (`ProductQueryController.java:53-77`)에 `sort` 와 `staleHours` 파라미터가 추가됐다.

- `sort`: "oldest" (마지막 수집이 오래된 순) 또는 기본값 "latest" (최근 수집 순). `ProductQueryService.search()` (124-137줄)가 `"oldest".equals(sort)` 일 때만 `merchantProductRepository.searchOldestFirst(...)` 를, 그 외엔 `search(...)` 를 호출한다 (내부적으로 정렬 방향만 다른 두 JPA 쿼리 메서드로 추정 — repository 구현 파일은 이번 조사 범위에서 직접 읽지 않았다).
- `staleHours`: 지정되면 `Instant.now().minus(Duration.ofHours(staleHours))` 를 `staleBoundary` 로 계산해 리포지토리 쿼리에 전달, **마지막 수집이 그 시각 이전인 상품만** 반환한다(126-127줄).
- `limit` 상한이 10,000까지 허용된 것도 이번 변경의 일부다 — "관리자 화면의 대량 재검증 대상 조회처럼 큰 값이 필요한 경우를 감안" (`ProductQueryController.java:45-46` 주석).

이 API가 바로 §9-5의 상품 재검증 화면이 "마지막 수집 오래된 순으로 N시간 이상 지난 상품만" 조회하는 데 쓰인다(`products-recheck.tsx` 의 `sort / staleHours` 필터 폼).

11. 관리자 대시보드 화면 ↔ API 매핑

화면	파일	호출 API	설명
현황	admin/collections/collections-dashboard.tsx, dashboard-data.ts	GET /internal/v1/dashboard/summary, GET /internal/v1/collection-jobs?page&size	최근 24시간(기본) job/task 집계, 판매처별 수집 상품 수, JSON/HTML 검증 일지율, 요청 이력 표
현황 → 재실행	같은 화면 표의 "재실행" 버튼	POST /api/collection-jobs/{jobId}/retry → POST /internal/v1/collection-jobs/{jobId}/retry	FAILED / PARTIAL job만 노출, 원래 조건 그대로 새 jobId 발행
새 수집 요청	admin/collections/new/new-collection-request-form.tsx	POST /api/collection-tasks → POST /internal/v1/collection-tasks/pages	판매처/검색어/페이지범위/필터 입력, 다중 페이지를 한 jobId로 등록
상품 재검증	admin/collections/products/page.tsx, products-recheck.tsx, products-data.ts	GET /internal/v1/products?merchant&sort&limit&staleHours (조회) / POST /api/offer-verifications/bulk → POST /internal/v1/offer-verifications/products/bulk (재검증 접수) / GET /internal/v1/offer-verifications/products/bulk/{batchId} (진행상태)	마지막 수집 오래된 순 정렬 + 경과 시간 필터로 대상 선정, 체크박스 또는 "전체" 버튼으로 일괄 재검증, 배치 진행률을 배너로 표시

공통 원칙: 모든 폼은 **fetch/XHR이 아니라 진짜 <form> GET/POST + 303 리다이렉트**로 동작한다 — 결과는 항상 URL 쿼리 파라미터(`?created=`, `?error=`, `?retried=`, `?retryError=`, `?batchId=`)로 전달되고 화면이 그것을 읽어 배너를 그린다. 대시보드/상품 재검증 페이지는 `export const dynamic = "force-dynamic"`으로 Next.js 캐시를 끄고 매 요청 서버에서 새로 가져온다.

12. Contract(JSON Schema) 요약

contracts/ 아래 서비스 간 공유 계약. Contract가 바뀌면 Schema, Go DTO, Java DTO, 테스트를 같은 작업에서 갱신하는 것이 원칙(Purchase_Research_Agent_시스템_구조.md §6).

Contract	방향	핵심 필드
contracts/collector/v1/search-request.schema.json	Backend → Collector	판매처, 검색어, 필터(가격/카테고리/사이즈/색상/재고/attributes)
contracts/collector/v1/collector-result.schema.json	Collector → Backend	requestId, operation, status(success/partial/failed), merchant, query, filters, totalCount, hasNext, collectedAt, collectorVersion, products[], warnings[], errors[] . 상품(products[])은 externalId, name, brand, categoryPath, productUrl, imageUrls, price{amount,currency}, shipping{fee,summary,provenance}, stockStatus, rating, reviewCount, options[], measurements, reviews[], provenance{sourceUrl,collectedAt,collectorVersion} , 선택적으로 verification{status,comparedFields,differences,jsonSourceUrl,htmlSourceUrl,verifiedAt}
contracts/collector/v1/verification-result.schema.json	Collector → Backend	구매 전 최신 정보와 변경 내용
contracts/collection/v1/collection-task.schema.json	Backend → RabbitMQ → Collector	schemaVersion, taskId, jobId, merchant, operation, priority, attempt, maxAttempts, requestedAt(tz 포함 ISO8601), idempotencyKey, payload{query,page,limit,locale,currency,filters}
contracts/collection/v1/collection-result.schema.json	Collector → RabbitMQ → Backend	schemaVersion, taskId, jobId, status(running/success/partial/failed), startedAt, completedAt, durationMs, collectorResult(collector-result.schema와 동일 구조 또는 null), error{code,message,retryable}

Python(app/messaging/contracts.py)과 Spring(CollectionResultEnvelope 등 Java DTO)이

둘 다 이 두 개의 collection/v1 Schema 파일을 직접 로드해 검증한다

(contracts.py:22-24 가 저장소 루트 기준 상대경로로 Schema 파일을 찾음) — Schema가 유일한 진실의 원천이고 언어별 DTO는 그 구현체다.

13. 한눈에 보는 상태값 정리

대상	상태값
collection_jobs.status	QUEUED → RUNNING/PROCESSING → COMPLETED \ PARTIAL \ FAILED
collection_tasks.status	QUEUED → RUNNING → SUCCESS \ PARTIAL \ FAILED \ PUBLISH_FAILED
CollectionResult(Queue) status	running → success \ partial \ failed
product_verifications.status	PENDING, MATCHED, MISMATCH, MISSING_IN_HTML, MISSING_IN_JSON, FAILED
OfferVerificationResponse status	QUEUED → RUNNING → VERIFIED \ CHANGED \ FAILED \ NOT_FOUND
BulkVerificationBatchResponse.status	PROCESSING → COMPLETED
CollectionRefreshResponse.DataStatus	MISSING, STALE, FRESH

14. 참고 문서

- 컴포넌트 단위 아키텍처 개요: [Purchase Research Agent 시스템 구조](#)
- DB 저장 설계의 배경(왜 snapshot인지, 왜 검색 컨텍스트를 분리했는지): [현재 수집 데이터와 DB 저장 흐름](#)
- 판매처 공통 수집 데이터 명세: [판매처 공통 수집 데이터 명세](#)
- Go Collector 변환 로직(Python과 별개의 전환 기준 구현): [Go Collector 데이터 변환 동작 설명](#)